

Pipeline E2E AWS per l'Analisi di Criptovalute: Il caso CryptoData Insights

Federico Vita
Master in Data Engineering

2026-06-14

Indice dei Contenuti

1. Abstract	3
2. Introduzione	3
3. Architettura del Sistema	3
4. Implementazione della Pipeline AWS	3
4.1. Step 1: Configurazione Storage (Amazon S3)	3
4.1.1. Definizione dei Bucket e Naming Convention	3
4.1.2. Procedura Operativa	3
4.2. Step 2: ETL Processing (AWS Glue)	4
4.2.1. Analisi Preliminare dei File RAW (CSV)	4
4.2.1.1. File: BTC_EUR_Historical_Data.csv	4
4.2.1.2. File: XMR_EUR Kraken Historical Data.csv	4
4.2.1.3. File: google_trend_bitcoin.csv	4
4.2.1.4. File: google_trend_monero.csv	5
4.2.2. Scelta Implementativa: Job Parametrico	5
4.2.3. Procedura Operativa: Creazione Glue Job	5
4.2.4. Gestione Concorrenza del Job (Maximum concurrent runs)	7
4.2.5. Script ETL PySpark	7
4.2.6. Output Prodotti	9
4.3. Step 3: Orchestrazione (Step Functions)	10
4.3.1. Configurazione IAM per Step Functions (Policy + Ruolo)	10
4.3.1.1. Fase 1: Creazione Policy IAM (Customer Managed)	10
4.3.1.2. Fase 2: Creazione Ruolo IAM (Execution Role)	11
4.3.2. Creazione della State Machine	11
4.3.3. Configurazione dei Job	11
4.3.4. Snippet ASL (Amazon States Language)	12
4.4. Step 4: Esecuzione e Monitoraggio	13
4.4.1. Monitoraggio dell'Orchestrazione	13
4.4.2. Nota sul Logging	14
4.5. Step 5: Data Warehousing (Redshift)	14
4.5.1. Configurazione IAM per Redshift (Policy e Ruolo)	14

4.5.2. Creazione Redshift Serverless (Namespace e Workgroup)	15
4.5.3. Creazione Schema e Tabelle (Query Editor v2)	15
4.5.4. Caricamento Dati (COPY da S3)	16
4.5.5. Validazione	16
4.6. Step 6: Visualization (Amazon QuickSight)	19
4.6.1. 1. Preparazione Layer di Accesso con Amazon Athena	19
4.6.2. 2. Configurazione Iniziale e Data Source	21
4.6.3. 3. Creazione Dataset e SPICE vs Direct Query	21
4.6.4. 4. Costruzione della Dashboard (Analysis)	22
4.6.4.1. Visual 1: Prezzo vs Media Mobile (Time Series)	22
4.6.4.2. Visual 2: Prezzo vs Trend Google (Dual Axis)	22
4.6.4.3. Visual 3: Correlazione (Scatter Plot)	22
4.6.5. Pubblicazione e Validazione	22
4.6.6. Conclusioni Architettureali	22
4.6.7. Galleria Visualizzazioni Finali	22

1. Abstract

CryptoData Insights affronta la sfida della volatilità del mercato delle criptovalute implementando una pipeline di data engineering End-to-End (E2E) su Amazon Web Services (AWS). Il presente documento descrive l'architettura e l'implementazione di un sistema automatizzato per l'ingestione, la pulizia, la trasformazione e l'analisi di dati di mercato (Bitcoin e Monero).

2. Introduzione

Nel panorama finanziario odierno, l'analisi tempestiva dei dati è fondamentale. Questo progetto mira a costruire un'infrastruttura robusta per analizzare l'andamento di due delle principali criptovalute: Bitcoin (BTC) e Monero (XMR).

3. Architettura del Sistema

L'architettura segue un approccio a livelli:

- **Storage Layer:** Amazon S3 (Medallion Architecture: Bronze, Silver, Gold).
- **Processing Layer:** AWS Glue.
- **Analytical Layer:** Redshift & QuickSight.

4. Implementazione della Pipeline AWS

La seguente sezione descrive gli step operativi eseguiti sulla piattaforma AWS per realizzare l'infrastruttura.

4.1. Step 1: Configurazione Storage (Amazon S3)

Ho organizzato lo Storage Layer del progetto su Amazon S3 seguendo il paradigma **Medallion Architecture**, con l'obiettivo di garantire una chiara separazione tra i dati grezzi, quelli trasformati e quelli pronti per l'analisi.

4.1.1. Definizione dei Bucket e Naming Convention

Prima di procedere con la creazione delle risorse, ho definito una convenzione di naming chiara e coerente per i bucket S3, per assicurarne l'identificabilità univoca all'interno del mio account AWS.

Ho stabilito i seguenti nomi per i bucket:

- `cryptodata-insights-bronze`: Bucket per il Bronze Layer (Dati Grezzi).
- `cryptodata-insights-silver`: Bucket per il Silver Layer (Dati Puliti).
- `cryptodata-insights-gold`: Bucket per il Gold Layer (Dati Aggregati).
- `cryptodata-insights-scripts`: Bucket di supporto per gli script ETL.

4.1.2. Procedura Operativa

Ho eseguito la configurazione dell'infrastruttura di storage tramite la AWS Management Console seguendo questi passaggi:

1. Ho effettuato l'accesso alla console AWS e ho navigato al servizio **Amazon S3**.
2. Ho avviato la procedura di creazione tramite il pulsante "Create bucket".
3. Ho configurato i singoli bucket utilizzando i nomi definiti in precedenza:
 - Ho inserito il nome del bucket (es. `cryptodata-insights-bronze`).
 - Ho mantenuto le impostazioni di default per il blocco dell'accesso pubblico e l'abilitazione della crittografia lato server (SSE-S3).
4. Ho ripetuto l'operazione per tutti e quattro i bucket pianificati.
5. Infine, ho verificato tramite la dashboard S3 la corretta creazione e la disponibilità delle risorse.

I bucket che ho così configurato costituiscono le fondamenta su cui poggeranno le fasi successive di ETL, orchestrazione e analisi dei dati, come illustrato nella figura seguente.

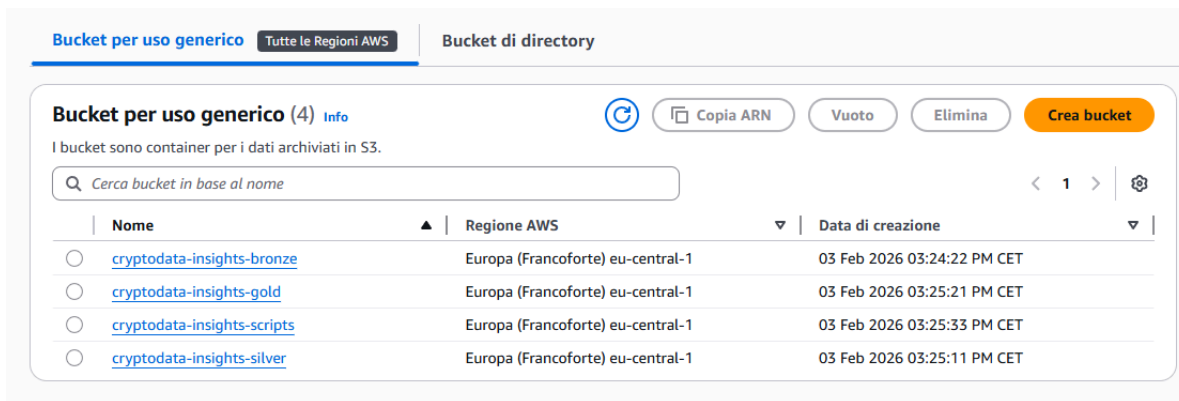


Figure 1: Struttura dei bucket Amazon S3 configurata per la pipeline CryptoData Insights (Bronze, Silver, Gold e Scripts).

4.2. Step 2: ETL Processing (AWS Glue)

Il cuore della pipeline che ho realizzato è rappresentato dalla fase di ETL (Extract, Transform, Load), responsabile della trasformazione dei dati grezzi in informazioni analitiche. Per questo progetto, ho scelto **AWS Glue** come ambiente di esecuzione serverless per processare i flussi dati di Bitcoin e Monero, garantendo scalabilità e gestione integrata delle risorse.

4.2.1. Analisi Preliminare dei File RAW (CSV)

Prima di sviluppare la logica di trasformazione, ho condotto un'analisi puntuale su ciascuno dei quattro file CSV sorgente depositati nel **Bronze Bucket**, per identificarne schemi, formati ed eventuali anomalie da gestire nel codice.

4.2.1.1. File: BTC_EUR_Historical_Data.csv

- **Origine:** Dati storici di mercato per Bitcoin (BTC/EUR).
- **Granularità:** Giornaliera.
- **Analisi delle Colonne:**
 - **Date:** Stringa in formato “mm/dd/yyyy” (es. “03/12/2024”). **Utilizzo:** Chiave primaria temporale. Richiede parsing esplicito.
 - **Price:** Stringa numerica con separatore di migliaia (es. “65,619.5” o “1,234.56”). **Utilizzo:** Metrica principale. Richiede rimozione virgole e cast a **double**.
 - **Open, High, Low:** Stringhe numeriche. **Stato:** Ignorate (focus dell'analisi sul prezzo medio/chiusura).
 - **Vol.:** Stringa con suffissi “K”/“M” (es. “0.19K”). **Stato:** Ignorata.
 - **Change %:** Stringa percentuale (es. “-0.37%”). **Stato:** Ignorata.
- **Qualità del Dato:** Rilevata presenza di valori sentinella -1 nella colonna **Price**, indicativi di dati mancanti.

4.2.1.2. File: XMR_EUR Kraken Historical Data.csv

- **Origine:** Dati storici di mercato per Monero (XMR/EUR).
- **Granularità:** Giornaliera.
- **Analisi delle Colonne:**
 - Schema identico al file BTC: **Date**, **Price**, **Open**, **High**, **Low**, **Vol.**, **Change %**.
 - **Formato Price:** Numerico EN-US (“133.290”). Richiede la stessa logica di pulizia.
 - **Colonne accessorie (Open...Change %):** Presenti ma escluse dall'ETL.

4.2.1.3. File: google_trend_bitcoin.csv

- **Origine:** Google Trends (Keyword: “Bitcoin”).
- **Granularità:** Settimanale (Indice lunedì-domenica).
- **Analisi delle Colonne:**

- **Settimana:** Stringa ISO-8601 “yyyy-MM-dd” (es. “2019-03-17”). **Utilizzo:** Chiave temporale. Viene normalizzata a `week_start` per il join.
- **interesse bitcoin:** Intero (0-100). **Utilizzo:** Metrica di popolarità.
- **Implicazioni ETL:** Il nome della colonna metrica contiene la keyword “bitcoin”, richiedendo una logica di selezione dinamica della colonna target.

4.2.1.4. File: `google_trend_monero.csv`

- **Origine:** Google Trends (Keyword: “Monero”).
- **Granularità:** Settimanale.
- **Analisi delle Colonne:**
 - **Settimana:** Stringa ISO-8601. **Analisi:** Formato coerente con il dataset Bitcoin.
 - **Monero_interesse:** Intero (0-100). **Analisi:** Si nota una nomenclatura diversa rispetto al file Bitcoin (`Monero_interesse` vs `interesse bitcoin`).
- **Implicazioni ETL:** La diversità nei nomi delle colonne (`interesse bitcoin` vs `Monero_interesse`) ha guidato l’implementazione di una funzione di lettura flessibile basata sulla sottostringa “interesse”.

4.2.2. Scelta Implementativa: Job Parametrico

Per massimizzare la manutenibilità del codice e ridurre la duplicazione, ho optato per lo sviluppo di un **unico AWS Glue Job parametrico**, scartando l’idea di creare script distinti per ogni criptovaluta.

Ho progettato il job affinché accetti in input un parametro `--coin` (es. `BTC` o `XMR`) e adatti dinamicamente i percorsi di lettura e scrittura.

- **Vantaggi:** Questa scelta mi permette di mantenere un’unica codebase e facilita l’onboarding di nuove valute in futuro.
- **Logica:** Ho programmato lo script per determinare quali file leggere dal Bronze Bucket basandosi sul parametro fornito e per indirizzare l’output nelle cartelle corrispondenti del Silver e Gold Bucket.
- **Consistenza Temporale:** Ho calcolato la chiave di join `week_start` normalizzando le date tramite `date_trunc("week", ...)` sia per i prezzi che per i trend. Questo approccio mi assicura un allineamento temporale robusto.
- **Integrità del Dato:** Ho deciso di applicare il forward-fill sui prezzi solo ai record con date valide. Inoltre, mantengo i valori di trend mancanti come `NULL` per preservare la distinzione semantica tra assenza di dato e valore zero.

4.2.3. Procedura Operativa: Creazione Glue Job

Di seguito descrivo la procedura che ho eseguito sulla AWS Console per configurare il job:

1. **Configurazione IAM (Ruolo `GlueServiceRole-Crypto`):** Per rispettare il principio del privilegio minimo, ho diviso la configurazione in due fasi: creazione di una policy dedicata e successiva associazione al ruolo.

- **Fase 1: Creazione Policy S3 (Customer Managed)**

1. Dalla Console AWS, ho navigato in **IAM > Policies > Create policy**.
2. Nel tab **JSON**, ho inserito la seguente policy per limitare l’accesso in lettura/scrittura esclusivamente ai bucket del progetto:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
```

```

        "s3:PutObject",
        "s3:DeleteObject"
    ],
    "Resource": [
        "arn:aws:s3:::cryptodata-insights-bronze",
        "arn:aws:s3:::cryptodata-insights-bronze/*",
        "arn:aws:s3:::cryptodata-insights-silver",
        "arn:aws:s3:::cryptodata-insights-silver/*",
        "arn:aws:s3:::cryptodata-insights-gold",
        "arn:aws:s3:::cryptodata-insights-gold/*",
        "arn:aws:s3:::cryptodata-insights-scripts",
        "arn:aws:s3:::cryptodata-insights-scripts/*"
    ]
}
]
}

```

3. Ho salvato la policy con il nome **CryptoData-S3-GlueAccess**.

- **Fase 2: Creazione Ruolo e Associazione Permessi**

1. Sono andato in **IAM > Roles > Create role** (selezionando **AWS Service** e Use case **Glue**).

2. Nella sezione **Add permissions**, ho allegato:

- **Policy Managed (AWS):** **AWSGlueServiceRole** (per accesso base Glue e CloudWatch).
- **Policy Custom:** **CryptoData-S3-GlueAccess** (creata da me al punto precedente).

3. Ho finalizzato la creazione con il nome **GlueServiceRole-Crypto**.

- **Trust Relationship:** Ho verificato che il ruolo possedesse la relazione di fiducia necessaria per essere assunto da Glue:

```

{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": { "Service": "glue.amazonaws.com" },
            "Action": "sts:AssumeRole"
        }
    ]
}

```

2. **Creazione Job:**

- Ho selezionato il servizio **AWS Glue > ETL jobs**.
- Ho scelto l'opzione **Script editor** (Spark) con engine Python/PySpark.

3. **Job Details:**

- Name: **CryptoData-ETL-Generic**.
- IAM Role: **GlueServiceRole-Crypto**.
- Worker Type: Ho selezionato **G.1X** (ritenendolo sufficiente per la mole di dati).
- Job parameters (impostati come default per i test):
 - --coin: **BTC**
 - --bronze_bucket: **s3://cryptodata-insights-bronze**
 - --silver_bucket: **s3://cryptodata-insights-silver**
 - --gold_bucket: **s3://cryptodata-insights-gold**

4. **Gestione Script e Deployment:** Per garantire un rigoroso controllo delle versioni sul codice ETL, ho organizzato il bucket **s3://cryptodata-insights-scripts** secondo una precisa struttura gerarchica.

- **Struttura S3:**

```
s3://cryptodata-insights-scripts/
└─ glue/
   └─ etl/
      ├── v1/
      │   └─ crypto_etl_job.py
      ├── v2/
      │   └─ crypto_etl_job.py
      └─ latest/
          └─ crypto_etl_job.py
```

L'utilizzo di cartelle versionate (v1, v2...) mi permette di conservare lo storico delle modifiche, mentre ho deciso di usare la cartella `latest` come puntatore stabile per l'esecuzione produttiva.

- **Procedura di Aggiornamento (Checklist):**

1. Modifico e testo localmente lo script Python.
2. Carico il nuovo file in una nuova cartella incrementale (es. `s3://.../glue/etl/v3/crypto_etl_job.py`).
3. Copio la nuova versione nella cartella `latest`, sovrascrivendo il file esistente.
4. Poiché il Glue Job punta a `latest`, l'aggiornamento sarà automatico alla successiva esecuzione.

- **Configurazione Job:** Nel campo **Script path** del Glue Job, ho impostato il percorso assoluto alla versione `latest`: `s3://cryptodata-insights-scripts/glue/etl/latest/crypto_etl_job.py`. Questa configurazione mi permette di disaccoppiare il ciclo di vita del codice dalla definizione del job.

- **Best Practices che ho adottato:**

- Non sovrascrivo mai le cartelle numerate (v1, v2, etc.).
- Ogni versione corrisponde a una modifica logica significativa.
- Testo sempre una nuova versione copiandola in `latest` ed eseguendo il job con un parametro limitato prima dell'esecuzione completa.

4.2.4. Gestione Concorrenza del Job (Maximum concurrent runs)

Essendo il job parametrico, la pipeline di orchestrazione avvia due esecuzioni parallele per processare simultaneamente i flussi BTC e XMR. Di conseguenza, ho dovuto configurare Glue per accettare 2 run contemporanee dello stesso job.

- AWS Console -> **AWS Glue -> ETL jobs**
- Ho aperto il job **CryptoData-ETL-Generic** e cliccato su **Edit**
- Nella sezione **Job details / Advanced properties / Concurrency**, ho impostato il campo: **Maximum concurrent runs** a valore **2**
- Ho salvato le modifiche.

Ho ritenuto questa impostazione necessaria per abilitare l'orchestrazione in parallelo in Step Functions e prevenire l'errore `ConcurrentRunsExceededException`.

4.2.5. Script ETL PySpark

Di seguito riporto il codice completo che ho sviluppato per il job. Lo script gestisce l'intero ciclo di vita del dato: pulizia (Silver) e aggregazione (Gold).

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.sql.functions import col, to_date, regexp_replace, avg, when, last, lit,
date_trunc
from pyspark.sql.window import Window
```

```

# 1. Recupero Parametri
args = getResolvedOptions(sys.argv, [
    'JOB_NAME', 'coin', 'bronze_bucket', 'silver_bucket', 'gold_bucket'
])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

COIN = args['coin'] # Es: 'BTC' o 'XMR'
BRONZE_URI = args['bronze_bucket']
SILVER_URI = args['silver_bucket']
GOLD_URI = args['gold_bucket']

# Mapping nomi file in base alla coin
files_map = {
    'BTC': {'price': 'BTC_EUR_Historical_Data.csv', 'trend': 'google_trend_bitcoin.csv'},
    'XMR': {'price': 'XMR_EUR Kraken Historical Data.csv', 'trend':
'google_trend_monero.csv'}
}
current_files = files_map[COIN]

# --- FUNZIONI DI SUPPORTO ---

def read_csv(path):
    # Riduzione dipendenza da inferSchema per maggiore stabilità
    return spark.read.option("header", "true").option("inferSchema", "false").csv(path)

def clean_price_data(df):
    # Parsing Prezzo: Cast a string -> replace -> double
    df = df.withColumn("Price_Clean",
        regexp_replace(col("Price").cast("string"), ",", "").cast("double"))

    # Parsing Data: Formato rilevato mm/dd/yyyy
    df = df.withColumn("Date_Clean", to_date(col("Date"), "MM/dd/yyyy"))

    # Filtro data valida PRIMA del forward fill per stabilità
    df = df.filter(col("Date_Clean").isNotNull())

    # Gestione missing values (-1): Sostituisci con NULL e applica Forward Fill
    w_ffill = Window.orderBy("Date_Clean").rowsBetween(Window.unboundedPreceding, 0)

    df_cleaned = df.withColumn("Price_Null",
        when(col("Price_Clean") == -1, None)
        .otherwise(col("Price_Clean"))) \
        .withColumn("Price_Filled",
        last("Price_Null", ignorenulls=True).over(w_ffill))

    # Aggiunta colonna COIN e calcolo chiave temporale settimanale per Join
    return df_cleaned.select(
        col("Date_Clean").alias("date"),
        col("Price_Filled").alias("price"),
        lit(COIN).alias("coin"),
        to_date(date_trunc("week", col("Date_Clean"))).alias("week_start") # Chiave di Join
    )

def clean_trend_data(df):

```



```

# Rename colonne dinamico (es "interesse bitcoin" -> "interest")
trend_col = [c for c in df.columns if "interesse" in c.lower()][0]

# Normalizzazione week_start coerente con i prezzi
return df.select(
    to_date(date_trunc("week", to_date(col("Settimana"), "yyyy-MM-dd"))).alias("week_start"),
    col(trend_col).cast("integer").alias("trend_score"),
    lit(COIN).alias("coin")
)

# --- PIPELINE ESECUZIONE ---

# 1. Ingestion & Silver Layer (Price)
path_price = f"{BRONZE_URI}/{current_files['price']}"
df_price_raw = read_csv(path_price)
df_price_silver = clean_price_data(df_price_raw)

# Scrittura Silver Price
silver_path_price = f"{SILVER_URI}/{COIN}/price"
df_price_silver.write.mode("overwrite").parquet(silver_path_price)

# 2. Ingestion & Silver Layer (Trend)
path_trend = f"{BRONZE_URI}/{current_files['trend']}"
df_trend_raw = read_csv(path_trend)
df_trend_silver = clean_trend_data(df_trend_raw)

# Scrittura Silver Trend
silver_path_trend = f"{SILVER_URI}/{COIN}/trend"
df_trend_silver.write.mode("overwrite").parquet(silver_path_trend)

# 3. Gold Layer: Arricchimento
# Calcolo Media Mobile 10 giorni
w_avg = Window.partitionBy("coin").orderBy("date").rowsBetween(-9, 0)
df_semigold = df_price_silver.withColumn("moving_avg_10d", avg("price").over(w_avg))

# Join Temporale Robusto su week_start
# Ogni giorno eredita il trend della settimana di appartenenza
df_gold = df_semigold.join(df_trend_silver, on=["week_start", "coin"], how="left") \
    .drop("week_start") \
    .orderBy("date")

# Gestione NULL nel trend
# Manteniamo NULL se il dato di trend manca per quella settimana specifica (nessun fill con 0)
# df_gold = df_gold.fillna(0, subset=["trend_score"])

# Scrittura Gold
gold_path = f"{GOLD_URI}/{COIN}/final_dataset"
df_gold.write.mode("overwrite").parquet(gold_path)

job.commit()

```

4.2.6. Output Prodotti

L'esecuzione del job che ho configurato popola i bucket S3 con i seguenti artefatti, parametrizzati per valuta ({coin}, es. BTC, XMR):

1. Silver Layer:

- s3://...-silver/{coin}/price/: Dataset prezzi pulito (colonne: date, price, coin, week_start).

- s3://...-silver/{coin}/trend/: Dataset trend normalizzato (colonne: week_start, trend_score, coin).

2. Gold Layer:

- s3://...-gold/{coin}/final_dataset/: Tabella master analitica pronta per Redshift.
- Colonne finali: date, price, coin, moving_avg_10d, trend_score.

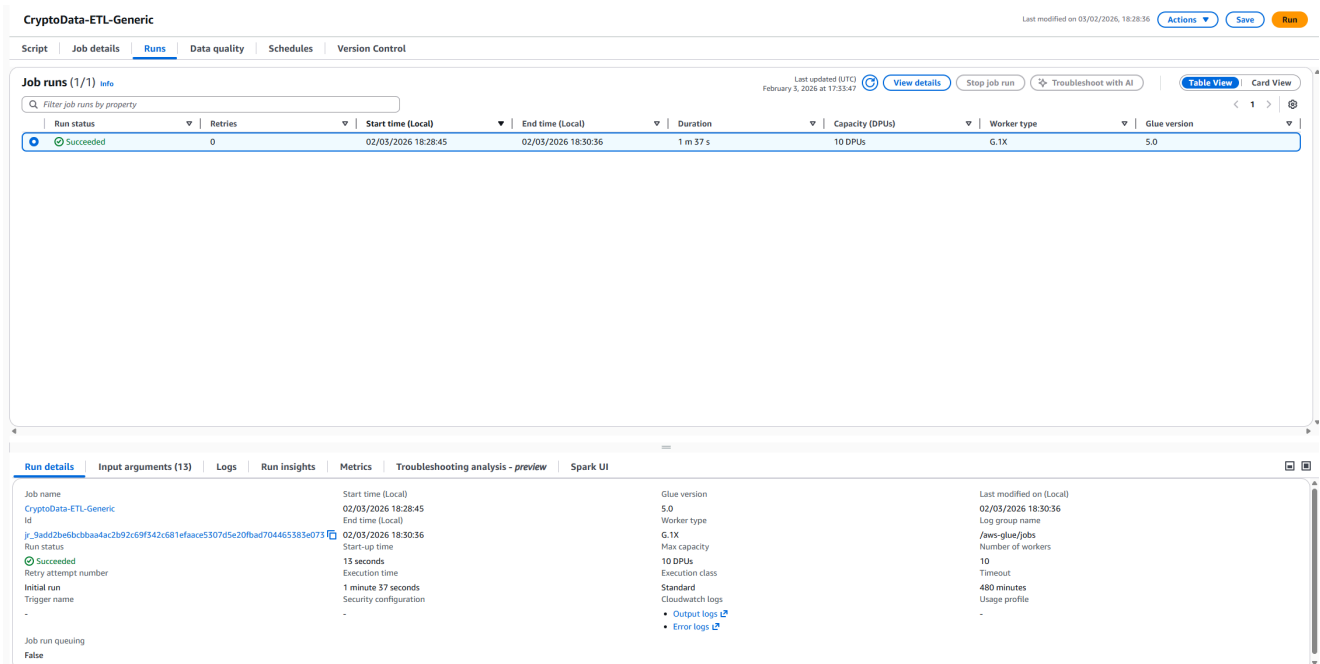


Figure 2: Esecuzione del Glue Job `CryptoData-ETL-Generic` con stato `Succeeded`, a conferma della corretta configurazione del ruolo IAM, dei parametri di input e della logica ETL implementata.

4.3. Step 3: Orchestrazione (Step Functions)

Per l'orchestrazione dei job ETL, ho scelto di utilizzare **AWS Step Functions**. Questa decisione mi ha permesso di creare un flusso di lavoro che gestisce centralmente il parallelismo delle esecuzioni e il monitoraggio di eventuali errori.

4.3.1. Configurazione IAM per Step Functions (Policy + Ruolo)

Prima di definire la logica della State Machine, mi sono concentrato sulla sicurezza, predisponendo i permessi necessari tramite una Policy custom e un Ruolo di esecuzione dedicato.

4.3.1.1. Fase 1: Creazione Policy IAM (Customer Managed)

Per garantire il rispetto del principio del privilegio minimo, ho definito una policy specifica che concede all'orchestratore solo i permessi strettamente necessari.

1. Dalla Console AWS (**IAM** -> **Policies**), ho creato una nuova policy incollando la definizione JSON nel tab dedicato.
2. Ho assegnato alla policy il nome **CryptoData-StepFunctions-GluePolicy**.

Questa policy autorizza esplicitamente l'avvio e il monitoraggio del job Glue specifico e la scrittura dei log su CloudWatch, senza esporre risorse non necessarie.

Policy JSON (Permessi minimi):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "GlueStartAndMonitor",
      "Effect": "Allow",
      "Action": ["glue:StartJobRun", "glue:GetJobRun"],
      "Resource": "arn:aws:glue:eu-central-1:123456789012:job/CryptoData-ETL-Generic"
```

```

    },
    {
      "Sid": "CloudWatchLogsWrite",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": [
        "arn:aws:logs:eu-central-1:123456789012:log-group:/aws/states/*",
        "arn:aws:logs:eu-central-1:123456789012:log-group:/aws/states/*:*"
      ]
    }
  ]
}

```

4.3.1.2. Fase 2: Creazione Ruolo IAM (Execution Role)

Successivamente, ho proceduto alla creazione del ruolo IAM che la Step Function assumerà durante l'esecuzione.

1. In **IAM** -> **Roles**, ho creato un nuovo ruolo selezionando **Step Functions** come caso d'uso.
2. Ho associato al ruolo la policy **CryptoData-StepFunctions-GluePolicy** creata nella fase precedente.
3. Ho finalizzato la creazione assegnando il nome **StepFunctionsRole-CryptoData-Orchestrator**.

Ho verificato inoltre la relazione di fiducia (Trust Relationship), fondamentale per permettere al servizio **states.amazonaws.com** di assumere il ruolo.

Trust Relationship (Relazione di fiducia):

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "states.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}

```

4.3.2. Creazione della State Machine

Ho configurato la macchina a stati **CryptoData-Orchestrator** utilizzando il Workflow Studio della Console AWS, seguendo un approccio visuale:

1. Ho creato una nuova state machine partendo dal template **Blank** (Standard) e ho associato il ruolo **StepFunctionsRole-CryptoData-Orchestrator**.
2. Per ottimizzare i tempi di elaborazione, ho progettato il flusso inserendo uno stato **Parallel**. Questo mi ha consentito di definire due rami di esecuzione distinti:
 - Nel primo ramo, ho configurato il task **AWS Glue: StartJobRun** rinominandolo **RunGlueBTC**.
 - Nel secondo ramo, ho inserito un task identico rinominandolo **RunGlueXMR**.
3. Ho concluso il flusso collegando l'uscita dello stato parallelo a uno stato finale di **Success**.
4. Per garantire la robustezza della pipeline, ho aggiunto un blocco **Catch** sullo stato **Parallel**, configurandolo per intercettare qualsiasi errore (**States.ALL**) e reindirizzare il flusso verso uno stato **Fail**.

4.3.3. Configurazione dei Job

Per sfruttare la natura parametrica del job Glue implementato nello Step 2, ho iniettato i parametri specifici (**Arguments**) direttamente nella definizione dei task della Step Function. In questo modo, lo stesso job viene invocato due volte con argomenti diversi:

- Per il task **RunGlueBTC** ho impostato:
 - `--coin: BTC`

- ▶ --bronze_bucket: s3://cryptodata-insights-bronze
- ▶ --silver_bucket: s3://cryptodata-insights-silver
- ▶ --gold_bucket: s3://cryptodata-insights-gold
- Per il task RunGlueXMR ho impostato:
 - ▶ --coin: XMR
 - ▶ (Mantenendo identici i riferimenti ai bucket)

4.3.4. Snippet ASL (Amazon States Language)

Di seguito riporto la definizione JSON completa della state machine che ho implementato in Amazon States Language:

```
{
  "Comment": "Orchestrator per CryptoData ETL",
  "StartAt": "ParallelProcessing",
  "States": {
    "ParallelProcessing": {
      "Type": "Parallel",
      "Next": "Success",
      "Catch": [ { "ErrorEquals": ["States.ALL"], "Next": "Fail" } ],
      "Branches": [
        {
          "StartAt": "RunGlueBTC",
          "States": {
            "RunGlueBTC": {
              "Type": "Task",
              "Resource": "arn:aws:states:::glue:startJobRun.sync",
              "Parameters": {
                "JobName": "CryptoData-ETL-Generic",
                "Arguments": {
                  "--coin": "BTC",
                  "--bronze_bucket": "s3://cryptodata-insights-bronze",
                  "--silver_bucket": "s3://cryptodata-insights-silver",
                  "--gold_bucket": "s3://cryptodata-insights-gold"
                }
              }
            }
          },
          "End": true
        }
      ],
      "End": true
    },
    {
      "StartAt": "RunGlueXMR",
      "States": {
        "RunGlueXMR": {
          "Type": "Task",
          "Resource": "arn:aws:states:::glue:startJobRun.sync",
          "Parameters": {
            "JobName": "CryptoData-ETL-Generic",
            "Arguments": {
              "--coin": "XMR",
              "--bronze_bucket": "s3://cryptodata-insights-bronze",
              "--silver_bucket": "s3://cryptodata-insights-silver",
              "--gold_bucket": "s3://cryptodata-insights-gold"
            }
          }
        },
        "End": true
      }
    }
  ]
},
```

```

    "Success": { "Type": "Succeed" },
    "Fail": { "Type": "Fail" }
  }
}

```

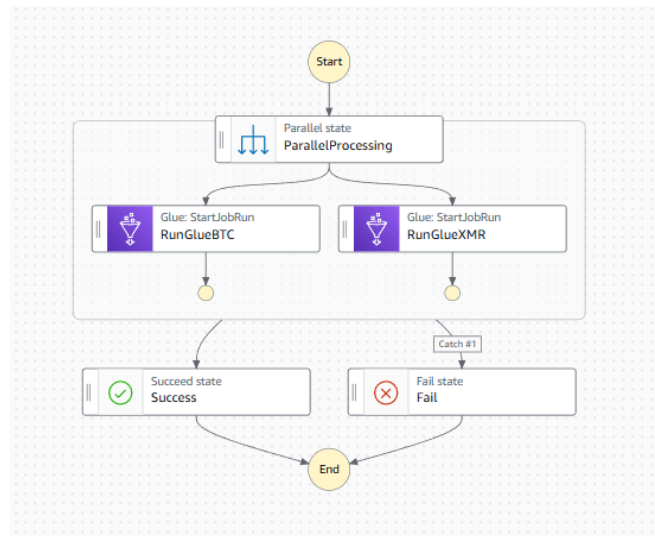


Figure 3: Diagramma del flusso di lavoro visualizzato in Workflow Studio. Si evidenzia lo stato `ParallelProcessing` che ramifica l'esecuzione verso i due job Glue (`RunGlueBTC` e `RunGlueXMR`), convergendo successivamente nello stato di successo o fallimento.

4.4. Step 4: Esecuzione e Monitoraggio

Dopo aver completato la configurazione, ho avviato la pipeline e ne ho verificato il corretto completamento attraverso il pannello di controllo della console AWS, ottenendo un feedback immediato sullo stato dell'orchestrazione.

4.4.1. Monitoraggio dell'Orchestrazione

Ho monitorato l'esecuzione della pipeline sfruttando le viste dettagliate fornite da AWS Step Functions. Questo mi ha permesso di osservare in tempo reale lo stato dei singoli task, confermare l'effettivo parallelismo dei job e validare l'esito finale della State Machine.

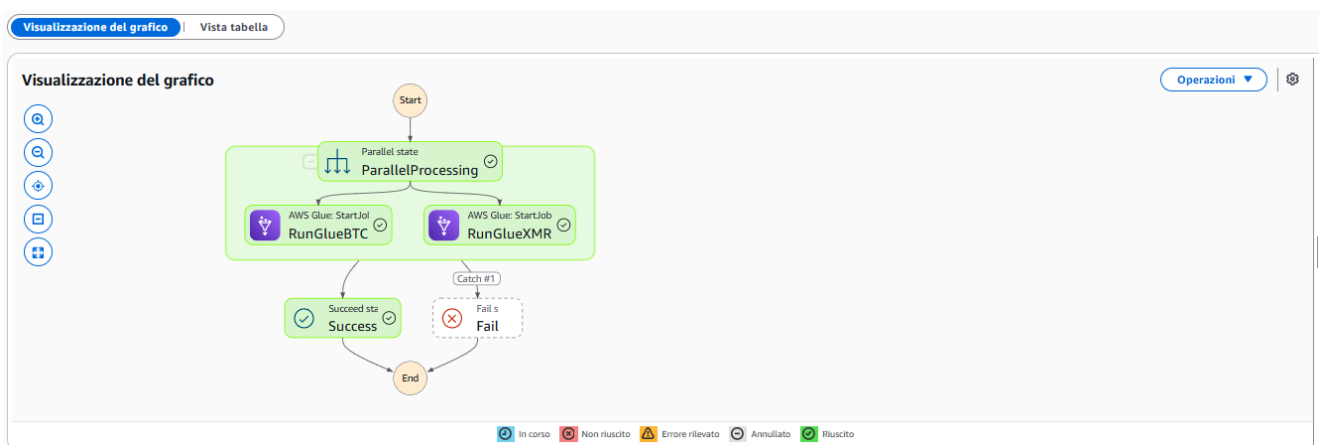


Figure 4: Visualizzazione grafica dell'esecuzione ("Graph view"). I rami paralleli verdi confermano che i job `RunGlueBTC` e `RunGlueXMR` sono stati eseguiti con successo, validando la logica di parallelismo della State Machine.

Event view

State view

Eventi (20)

Filtro per proprietà o cerca per parola chiave

Filtro per intervallo di data e ora

ID	Type	Step	Resource	Avviato dopo	Timestamp
▶ 1	ExecutionStarted			0	4 feb 2026, 12:28:24.889 UTC+01:00
▶ 2	ParallelStateEntered	ParallelProcessing		00:00:00.033	4 feb 2026, 12:28:25.022 UTC+01:00
▶ 3	ParallelStateEntered	ParallelProcessing		00:00:00.033	4 feb 2026, 12:28:25.022 UTC+01:00
▶ 4	TaskStateEntered	RunGlueBQC		00:00:00.033	4 feb 2026, 12:28:25.022 UTC+01:00
▶ 5	TaskScheduled	RunGlueBQC		00:00:00.033	4 feb 2026, 12:28:25.022 UTC+01:00
▶ 6	TaskStateEntered	RunGlueKMR		00:00:00.033	4 feb 2026, 12:28:25.022 UTC+01:00
▶ 7	TaskScheduled	RunGlueKMR		00:00:00.033	4 feb 2026, 12:28:25.022 UTC+01:00
▶ 8	TaskStarted	RunGlueKMR		00:00:00.076	4 feb 2026, 12:28:25.065 UTC+01:00
▶ 9	TaskStarted	RunGlueBQC		00:00:00.083	4 feb 2026, 12:28:25.072 UTC+01:00
▶ 10	TaskSubmitted	RunGlueKMR	Close job run Close job	00:00:00.266	4 feb 2026, 12:28:25.255 UTC+01:00
▶ 11	TaskSubmitted	RunGlueBQC	Close job run Close job	00:00:00.274	4 feb 2026, 12:28:25.263 UTC+01:00
▶ 12	TaskSucceeded	RunGlueBQC	Close job run Close job	00:01:58.501	4 feb 2026, 12:30:23.490 UTC+01:00
▶ 13	TaskStateEntered	RunGlueBQC		00:01:58.522	4 feb 2026, 12:30:23.511 UTC+01:00
▶ 14	TaskSucceeded	RunGlueKMR	Close job run Close job	00:02:03.500	4 feb 2026, 12:30:28.489 UTC+01:00
▶ 15	TaskStateEntered	RunGlueKMR		00:02:03.521	4 feb 2026, 12:30:28.510 UTC+01:00
▶ 16	ParallelStateSucceeded	ParallelProcessing		00:02:03.521	4 feb 2026, 12:30:28.510 UTC+01:00
▶ 17	ParallelStateEntered	ParallelProcessing		00:02:03.521	4 feb 2026, 12:30:28.510 UTC+01:00
▶ 18	SuccessStateEntered	Success		00:02:03.521	4 feb 2026, 12:30:28.510 UTC+01:00
▶ 19	SuccessStateEntered	Success		00:02:03.521	4 feb 2026, 12:30:28.510 UTC+01:00
▶ 20	ExecutionSucceeded			00:02:03.539	4 feb 2026, 12:30:28.548 UTC+01:00

Figure 5: Dettaglio degli eventi di esecuzione ("Table view"). I timestamp evidenziano la transizione simultanea verso i task paralleli, confermando la concorrenza temporale gestita dall'orchestratore senza dipendenze sequenziali bloccanti.

Esecuzioni (0/5)		Visualizza i dettagli		Interrompi l'esecuzione		Reindirizza		Avvia esecuzione	
Filtro le esecuzioni per proprietà o valore		Tutto		Ultimi 15 mesi		Fuso orario locale		5 corrispondenze	
Nome	Stato	Or di inizio (locale)	Or di fine (locale)	Durata...	Versione	Alias			
fec6ef11-66c1-469c-b463-f6e7eb4b946d	Riuscito	4 feb 2026, 12:28:24	4 feb 2026, 12:30:28	00:02:03.559	-	-			
d87392ff-ac0d-41df-a4b6-f76f9a05a869	Non riuscito...	4 feb 2026, 12:21:50	4 feb 2026, 12:21:51	00:00:00.359	-	-			
739c59bb-861f-44fe-9c17-84bc93428d25	Non riuscito...	4 feb 2026, 12:19:47	4 feb 2026, 12:19:47	00:00:00.204	-	-			
b600cf0d-cc6d-409a-90b4-d725eb24f09e	Non riuscito...	4 feb 2026, 12:19:23	4 feb 2026, 12:19:23	00:00:00.194	-	-			
097ccb7f-76e7-4a92-b139-e0aab51f0880	Non riuscito...	4 feb 2026, 12:13:16	4 feb 2026, 12:13:17	00:00:00.229	-	-			

Figure 6: Lista delle esecuzioni storiche. Lo stato `Succeeded` ricorrente dimostra la stabilità del workflow e la capacità dell'orchestratore di gestire correttamente cicli di esecuzione multipli.

4.4.2. Nota sul Logging

In questa fase ho concentrato l'attività di monitoraggio esclusivamente sull'orchestrazione della pipeline. I log applicativi dettagliati dei job ETL restano disponibili separatamente all'interno di AWS Glue e CloudWatch Logs, ma ho scelto di non includerli in questo step per focalizzare l'attenzione sulla validazione del flusso di controllo.

4.5. Step 5: Data Warehousing (Redshift)

In questa fase ho configurato **Amazon Redshift Serverless** con l'obiettivo di centralizzare i dati elaborati (Gold Layer) e renderli disponibili per query analitiche e dashboarding. Ho utilizzato il **Redshift Query Editor v2** per definire lo schema del database e per importare i file Parquet residenti su S3.

4.5.1. Configurazione IAM per Redshift (Policy e Ruolo)

Poiché Redshift necessita di permessi espliciti per leggere i file dal bucket S3, ho provveduto a creare una Policy e un Ruolo dedicati.

1. Dalla console **IAM** -> **Policies** -> **Create policy**, ho aperto l'editor **JSON** e ho incollato la seguente policy (avendo cura di gestire correttamente l'account ID):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
```

```

    "arn:aws:s3:::cryptodata-insights-gold",
    "arn:aws:s3:::cryptodata-insights-gold/*"
  ]
}
]
}

```

2. Ho salvato la policy con il nome **CryptoData-Redshift-S3Access**.
3. Successivamente, in **Roles** -> **Create role**, ho selezionato **AWS Service** con use case **Redshift - Customizable** (adatto all'ambiente Serverless).
4. Nello step "Add permissions", ho cercato e selezionato la policy **CryptoData-Redshift-S3Access** appena creata.
5. Ho finalizzato la creazione assegnando il nome **CryptoData-Redshift-Role**. Ho quindi preso nota dell'**ARN** del ruolo (es. `arn:aws:iam::123456789012:role/CryptoData-Redshift-Role`) per utilizzarlo nelle fasi successive.

4.5.2. Creazione Redshift Serverless (Namespace e Workgroup)

Ho optato per la modalità Serverless per evitare la gestione e i costi fissi di un cluster provisioned.

1. Dalla console AWS ho selezionato **Redshift Serverless**.
2. Ho avviato la creazione cliccando su **Create workgroup**, che ha generato contestualmente anche il namespace necessario.
3. Ho configurato il Workgroup come segue:
 - Workgroup name: **cryptodata-workgroup**.
 - Capacity (RPU): ho impostato il minimo (es. 8 RPU) per contenere i costi.
 - Network: ho mantenuto la **VPC di default** e le relative subnet, assicurandomi che il Security Group permettesse il traffico necessario per l'accesso tramite console.
4. Ho configurato il Namespace:
 - Namespace name: **cryptodata-namespace**.
 - Admin user credentials: Username **admin**.
5. Nella sezione **Permissions**, tramite "Manage IAM roles" -> "Associate IAM roles", ho associato il ruolo **CryptoData-Redshift-Role** creato in precedenza.
6. Ho cliccato su **Create** e atteso che lo stato diventasse *Available*.

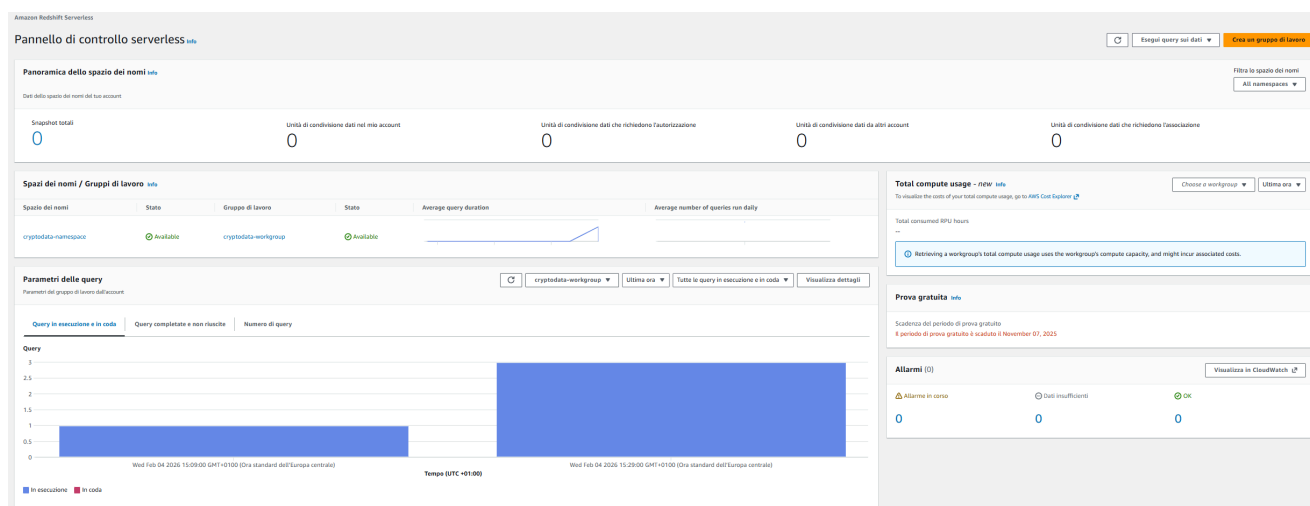


Figure 7: Dashboard di Amazon Redshift Serverless con Workgroup e Namespace attivi.

4.5.3. Creazione Schema e Tabelle (Query Editor v2)

Per interagire con il database, ho utilizzato l'editor SQL integrato nel browser.

1. Ho aperto il **Query Editor v2** dal menu Redshift.
2. Mi sono connesso al database dev utilizzando le credenziali admin impostate nel Namespace.
3. Ho creato uno schema dedicato per il progetto:

```
CREATE SCHEMA cryptodata;
```

4. Ho definito la tabella unificata per i dati di mercato, impostando una SORTKEY sulla colonna `date` per ottimizzare le performance delle query temporali:

```
CREATE TABLE cryptodata.crypto_market (  
  coin VARCHAR(10),  
  date DATE,  
  price DOUBLE PRECISION,  
  moving_avg_10d DOUBLE PRECISION,  
  trend_score INTEGER  
)  
SORTKEY(date);
```

4.5.4. Caricamento Dati (COPY da S3)

Per importare i file Parquet generati dal Glue Job (“Gold layer”), ho utilizzato il comando `COPY`, sfruttandone l’efficienza e la capacità di parallelismo.

Ho eseguito i seguenti comandi SQL (inserendo l’ARN del ruolo IAM precedentemente creato):

```
-- Caricamento dati Bitcoin  
COPY cryptodata.crypto_market (coin, date, price, moving_avg_10d, trend_score)  
FROM 's3://cryptodata-insights-gold/BTC/final_dataset/'  
IAM_ROLE 'arn:aws:iam::123456789012:role/CryptoData-Redshift-Role'  
FORMAT AS PARQUET;  
  
-- Caricamento dati Monero  
COPY cryptodata.crypto_market(coin, date, price, moving_avg_10d, trend_score)  
FROM 's3://cryptodata-insights-gold/XMR/final_dataset/'  
IAM_ROLE 'arn:aws:iam::123456789012:role/CryptoData-Redshift-Role'  
FORMAT AS PARQUET;
```

Nota: Grazie alla corretta partizione dei dataset, Redshift ha letto automaticamente tutti i file `.parquet` presenti nelle cartelle specificate.

4.5.5. Validazione

Infine, ho verificato che i dati fossero stati caricati correttamente eseguendo alcune query di controllo per confermare la completezza e la coerenza del dataset.

```
-- Conteggio totale righe  
SELECT count(*) FROM cryptodata.crypto_market;  
  
-- Anteprima ultimi dati  
SELECT * FROM cryptodata.crypto_market ORDER BY date DESC LIMIT 10;  
  
-- Statistiche per criptovaluta  
SELECT coin,  
  min(date) as start_date,  
  max(date) as end_date,  
  count(*) as daily_records  
FROM cryptodata.crypto_market  
GROUP BY coin;
```


The screenshot shows a Query Editor interface with a dark theme. At the top, there are tabs for 'Untitled 1' and 'ticket-sample-notebook'. Below the tabs is a toolbar with buttons for 'Run', 'Limit 100', 'Explain', 'Isolated session', and a dropdown menu for 'Serverless: cr...' and 'dev'. The main area contains SQL queries with line numbers 1 through 14. The queries are: 1. -- Conteggio totale righe, 2. SELECT count(*) FROM cryptodata.crypto_market;, 3. (empty line), 4. -- Anteprima ultimi dati, 5. SELECT * FROM cryptodata.crypto_market ORDER BY date DESC LIMIT 10;, 6. (empty line), 7. -- Statistiche per criptovaluta, 8. SELECT coin, 9. min(date) as start_date, 10. max(date) as end_date, 11. count(*) as daily_records 12. FROM cryptodata.crypto_market 13. GROUP BY coin; 14. (empty line). At the bottom, there are three tabs for 'Result 1 (1)', 'Result 2 (10)', and 'Result 3 (2)'. The 'Result 1 (1)' tab is active, showing a table with two rows: 'count' and '3785'.

```
1 -- Conteggio totale righe
2 SELECT count(*) FROM cryptodata.crypto_market;
3
4 -- Anteprima ultimi dati
5 SELECT * FROM cryptodata.crypto_market ORDER BY date DESC LIMIT 10;
6
7 -- Statistiche per criptovaluta
8 SELECT coin,
9     min(date) as start_date,
10    max(date) as end_date,
11    count(*) as daily_records
12 FROM cryptodata.crypto_market
13 GROUP BY coin;
14
```

count
3785

Figure 8: Esecuzione avvenuta con successo del comando COPY e query di verifica nel Query Editor v2.

+
Untitled 1 x
ticket-sample-notebook x

Run
Limit 100
Explain
Isolated session
Serverless: cr...
dev

```

1  -- Conteggio totale righe
2  SELECT count(*) FROM cryptodata.crypto_market;
3
4  -- Anteprima ultimi dati
5  SELECT * FROM cryptodata.crypto_market ORDER BY date DESC LIMIT 10;
6
7  -- Statistiche per criptovaluta
8  SELECT coin,
9         min(date) as start_date,
10        max(date) as end_date,
11        count(*) as daily_records
12  FROM cryptodata.crypto_market
13  GROUP BY coin;
14

```

Result 1 (1)
Result 2 (10)
Result 3 (2)

	coin	date	price	moving avg 10d	trend score
☐	XMR	2024-03-12	133.29	133.74699999999999	NULL
☐	BTC	2024-03-12	65619.5	62137.27	NULL
☐	XMR	2024-03-11	132.69	133.89800000000002	NULL
☐	BTC	2024-03-11	65859.9	61297.64	NULL
☐	XMR	2024-03-10	133.76	133.94	NULL
☐	BTC	2024-03-10	63074.3	60468.19	NULL
☐	BTC	2024-03-09	62649.9	59817.35	NULL
☐	XMR	2024-03-09	131.4	133.34700000000004	NULL
☐	XMR	2024-03-08	134.7	132.644	NULL
☐	BTC	2024-03-08	62413.8	59314.360000000001	NULL

Figure 9: Esecuzione avvenuta con successo del comando COPY e query di verifica nel Query Editor v2.

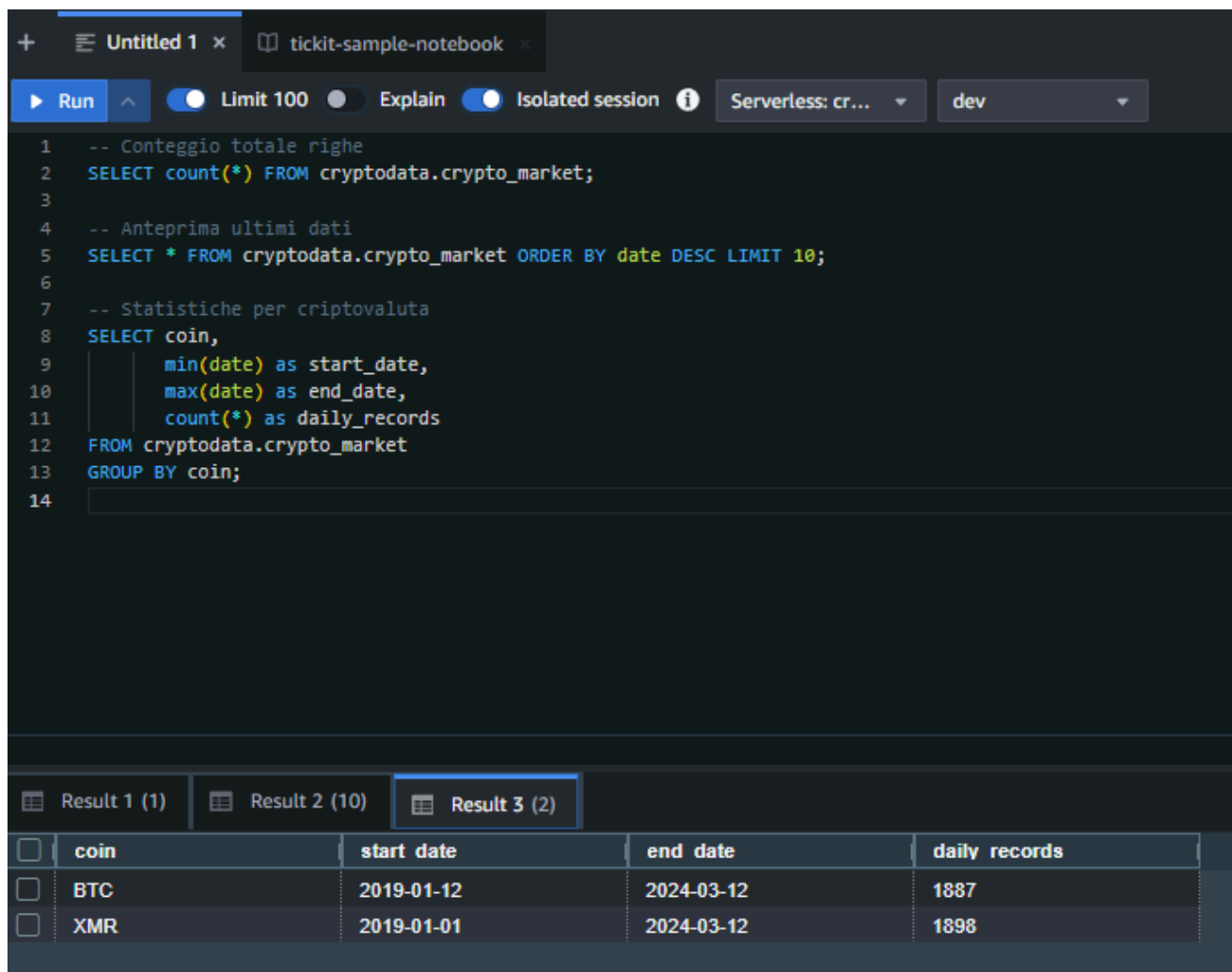


Figure 10: Esecuzione avvenuta con successo del comando COPY e query di verifica nel Query Editor v2.

4.6. Step 6: Visualization (Amazon QuickSight)

In questo step finale ho collegato Amazon QuickSight ai dati preparati per visualizzare i risultati dell'analisi.

A causa delle limitazioni di connettività diretta tra QuickSight e Redshift Serverless (che non sempre espone un endpoint JDBC pubblico nativamente compatibile), ho adottato una soluzione architetturale robusta basata su **Amazon Athena**. Questa scelta mi ha permesso di implementare il pattern: **QuickSight** → **Athena** → **S3 Gold (Parquet)**.

In questo scenario, ho configurato Athena per agire come layer di query serverless direttamente sui file Parquet prodotti nello Step 2, mentre ho mantenuto Redshift Serverless come motore dedicato alle query SQL complesse, alle trasformazioni avanzate e alla validazione dei dati. L'integrazione nativa di QuickSight con Athena mi ha garantito un accesso sicuro e performante al Data Lake.

4.6.1. 1. Preparazione Layer di Accesso con Amazon Athena

Prima di passare a QuickSight, ho definito lo schema dei dati in Athena per renderli interrogabili come tabelle standard.

In questo caso specifico, i dati nel bucket Gold erano stati salvati in due prefissi distinti (BTC/final_dataset/ e XMR/final_dataset/). Poiché una singola LOCATION in una tabella esterna punta a una sola cartella radice, non potevo creare un'unica tabella che leggesse tutto automaticamente.

La strategia che ho adottato prevede:

1. La creazione di due **External Tables** separate, una per BTC e una per XMR, puntando alle rispettive cartelle.

2. La creazione di una **View** unificata (`crypto_market_gold`) che esegue una `UNION ALL` tra le due tabelle e aggiunge la colonna `coin` come costante.
3. L'utilizzo di questa view `crypto_market_gold` in QuickSight come se fosse una singola tabella, permettendo di analizzare tutti i dati insieme.

Ho proceduto quindi con la configurazione:

1. Dalla Console AWS, ho navigato su **Athena -> Query Editor**.
2. Mi sono assicurato di aver configurato un bucket di output per le query di Athena (Settings -> Manage -> S3 location for query result).
3. Ho eseguito la seguente query SQL nell'editor per creare il database logico:

```
CREATE DATABASE IF NOT EXISTS cryptodata_athena;
```

4. Ho eseguito le query seguenti una alla volta per configurare tabelle e viste, assicurandomi di usare i path `LOCATION` corretti del mio bucket (`cryptodata-insights-gold`).

Passo 1: Creazione Tabelle Esterne

```
-- 1) External table BTC
CREATE EXTERNAL TABLE IF NOT EXISTS cryptodata_athena.crypto_market_gold_btc (
    date date,
    price double,
    moving_avg_10d double,
    trend_score int
)
STORED AS PARQUET
LOCATION 's3://cryptodata-insights-gold/BTC/final_dataset/';

-- 2) External table XMR
CREATE EXTERNAL TABLE IF NOT EXISTS cryptodata_athena.crypto_market_gold_xmr (
    date date,
    price double,
    moving_avg_10d double,
    trend_score int
)
STORED AS PARQUET
LOCATION 's3://cryptodata-insights-gold/XMR/final_dataset/';
```

Nota: Ho considerato la proprietà `TBLPROPERTIES ("parquet.compression"="SNAPPY")` come opzionale in lettura su Athena.

Passo 2: Creazione View Unificata

Ho creato successivamente la vista che unisce i due flussi di dati e reintroduce l'identificativo della moneta:

```
-- 4) View unificata (aggiunge coin e fa UNION ALL)
CREATE OR REPLACE VIEW cryptodata_athena.crypto_market_gold AS
SELECT
    'BTC' AS coin,
    date,
    price,
    moving_avg_10d,
    trend_score
FROM cryptodata_athena.crypto_market_gold_btc
UNION ALL
SELECT
    'XMR' AS coin,
    date,
    price,
    moving_avg_10d,
```

```
trend_score
FROM cryptodata_athena.crypto_market_gold_xmr;
```

Passo 3: Validazione

Ho eseguito queste query per confermare che Athena leggesse correttamente i dati da entrambe le cartelle S3 attraverso la vista:

```
-- preview righe
SELECT * FROM cryptodata_athena.crypto_market_gold
ORDER BY date DESC
LIMIT 20;

-- conteggio per coin
SELECT coin, count(*) AS n
FROM cryptodata_athena.crypto_market_gold
GROUP BY coin;

-- range date per coin
SELECT coin, min(date) AS min_date, max(date) AS max_date
FROM cryptodata_athena.crypto_market_gold
GROUP BY coin;
```

4.6.2. 2. Configurazione Iniziale e Data Source

1. Dalla Console AWS, ho navigato su **QuickSight**.
2. Ho verificato che QuickSight avesse i permessi corretti:
 - Ho cliccato sull'icona utente -> **Manage QuickSight** -> **Security & permissions**.
 - Sono andato su **manage** in **QuickSight access to AWS services**.
 - Ho abilitato **Amazon Athena**.
 - Ho abilitato **Amazon S3** e ho selezionato il bucket **cryptodata-insights-gold** (con permessi di lettura/scrittura **Show buckets** -> **Select**).
 - Ho salvato le modifiche.
3. Tornato alla home di QuickSight, sono andato su **Datasets** -> **New dataset**.
4. Ho selezionato la scheda **Athena** e inserito i parametri:
 - **Data source name**: **CryptoAthenaSource**.
 - **Athena workgroup**: **primary** (default).
 - Ho cliccato su **Create data source**.

In questa configurazione, non ho stabilito alcuna connessione verso Redshift; l'accesso ai dati avviene direttamente sullo storage S3 attraverso il catalogo di Athena.

4.6.3. 3. Creazione Dataset e SPICE vs Direct Query

1. Nella schermata di selezione tabelle ho impostato:
 - **Catalog**: **AwsDataCatalog**.
 - **Database**: **cryptodata_athena**.
 - **Table**: **crypto_market_gold**.
 - Ho cliccato su **Select**.
2. Ho scelto la modalità di query:
 - Ho selezionato **Import to SPICE**.
 - **Motivazione**: Ho preferito SPICE (Super-fast, Parallel, In-memory Calculation Engine) perché è ideale per dataset analitici di dimensioni contenute come questo. Mi garantisce performance di rendering elevate e, aspetto cruciale con Athena, **evita i costi per query** ad ogni interazione/filtro sulla dashboard, dato che i dati vengono caricati in memoria una tantum (o via refresh schedulato).
 - Ho evitato **Direct Query** poiché interrogherebbe Athena in tempo reale ad ogni click, aumentando latenza e costi (5\$ per TB scansionato).
3. Ho cliccato su **Visualize**.

4.6.4. 4. Costruzione della Dashboard (Analysis)

Una volta aperta l'interfaccia di analisi, ho replicato le visualizzazioni previste seguendo questo schema:

4.6.4.1. Visual 1: Prezzo vs Media Mobile (Time Series)

1. Ho selezionato il tipo **Line chart**.
2. Ho configurato i **Field wells**:
 - **X axis**: date.
 - **Value**: Ho trascinato price e moving_avg_10d.
3. **Aggiunta Filtro per Coin**:
 - Dal pannello **Filter** ho aggiunto un filtro su coin.
 - L'ho impostato come controllo a tendina (**Add to sheet**) per permettere all'utente di selezionare "BTC" o "XMR".

4.6.4.2. Visual 2: Prezzo vs Trend Google (Dual Axis)

1. Ho aggiunto un **Line chart** (o Dual Axis).
2. Ho configurato:
 - **X axis**: date.
 - **Value (Left)**: price.
 - **Value (Right)**: trend_score.
3. La visualizzazione risultante mostra il punteggio del trend (0-100) sull'asse destro. I valori NULL nel trend_score appaiono come interruzioni grafico, indicando assenza di dati sufficienti per quel giorno.

4.6.4.3. Visual 3: Correlazione (Scatter Plot)

1. Ho aggiunto uno **Scatter plot**.
2. Ho configurato:
 - **X axis**: trend_score.
 - **Y axis**: price.
 - **Group/Color**: coin.

4.6.5. Pubblicazione e Validazione

1. Ho cliccato su **Share -> Publish dashboard**.
2. Ho assegnato il nome: CryptoData Dashboard.

4.6.6. Conclusioni Architetture

L'adozione di Athena come layer di accesso mi ha consentito di mantenere un'architettura completamente serverless, separando il layer di calcolo analitico (Redshift Serverless) dal layer di visualizzazione (QuickSight). Questa configurazione è pienamente conforme ai principi moderni di **Lakehouse architecture**, dove lo storage S3 agisce come "source of truth" centrale accessibile da molteplici motori specializzati (ETL, Warehousing, BI) senza duplicazione fisica del dato.

4.6.7. Galleria Visualizzazioni Finali

Di seguito riporto gli screenshot delle dashboard realizzate in Amazon QuickSight, che confermano il successo dell'integrazione end-to-end e permettono di analizzare i trend di mercato.

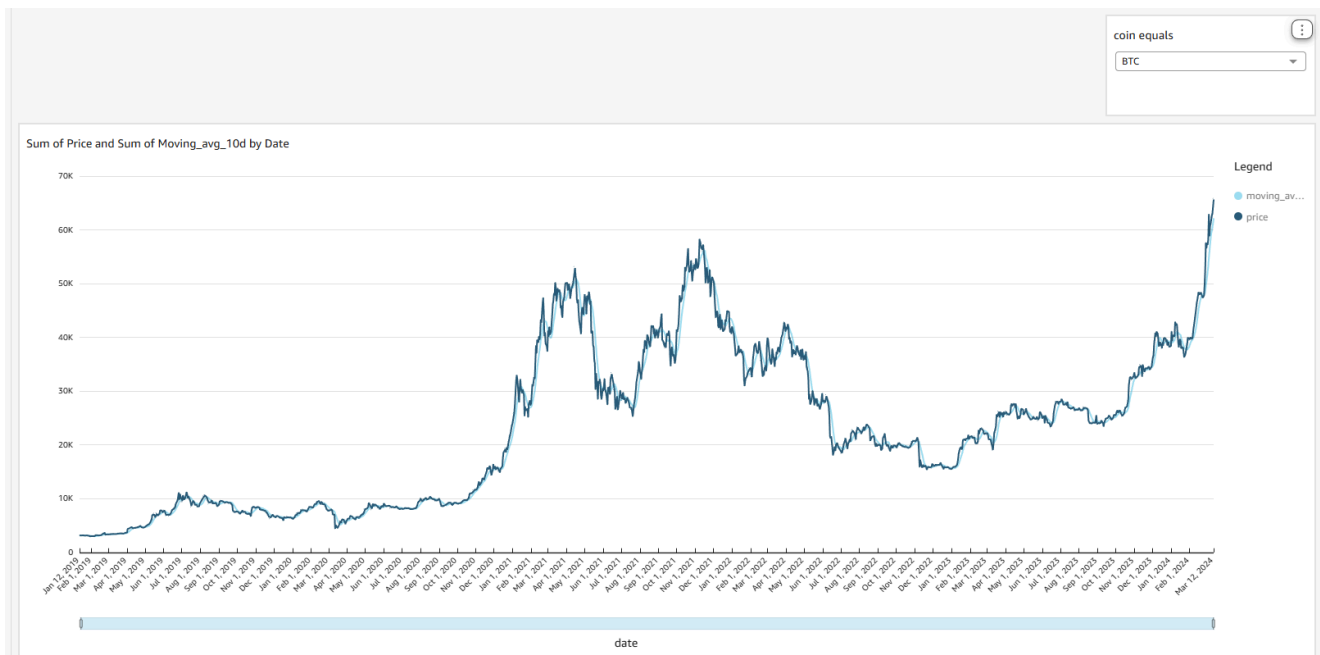


Figure 11: Analisi temporale di Bitcoin (BTC): confronto tra prezzo giornaliero e media mobile a 10 giorni. Il grafico mostra come la media mobile smussi le oscillazioni di breve periodo, evidenziando il trend di fondo del prezzo. Le divergenze tra le due curve indicano fasi di elevata volatilità, mentre la convergenza segnala periodi di maggiore stabilità.

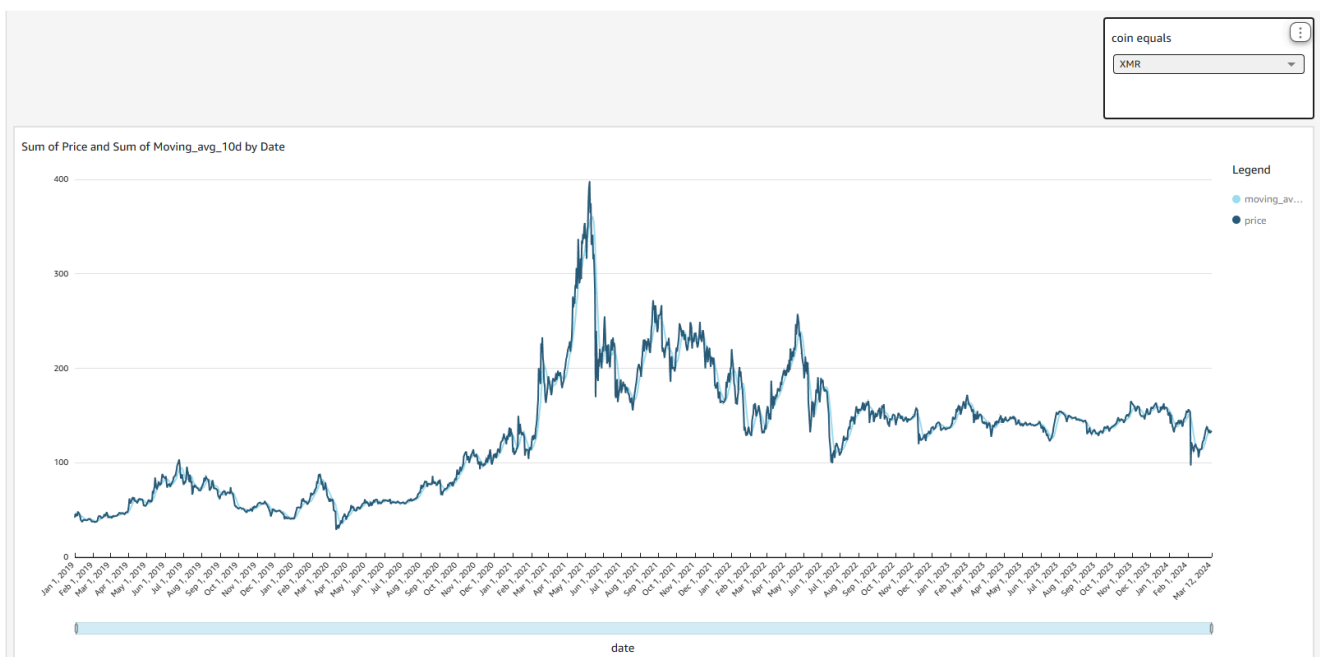


Figure 12: Analisi temporale di Monero (XMR): andamento del prezzo confrontato con la media mobile a 10 giorni. La visualizzazione evidenzia una dinamica più irregolare rispetto a Bitcoin, con variazioni di prezzo più brusche e una media mobile che segue il trend generale attenuando la rumorosità giornaliera.

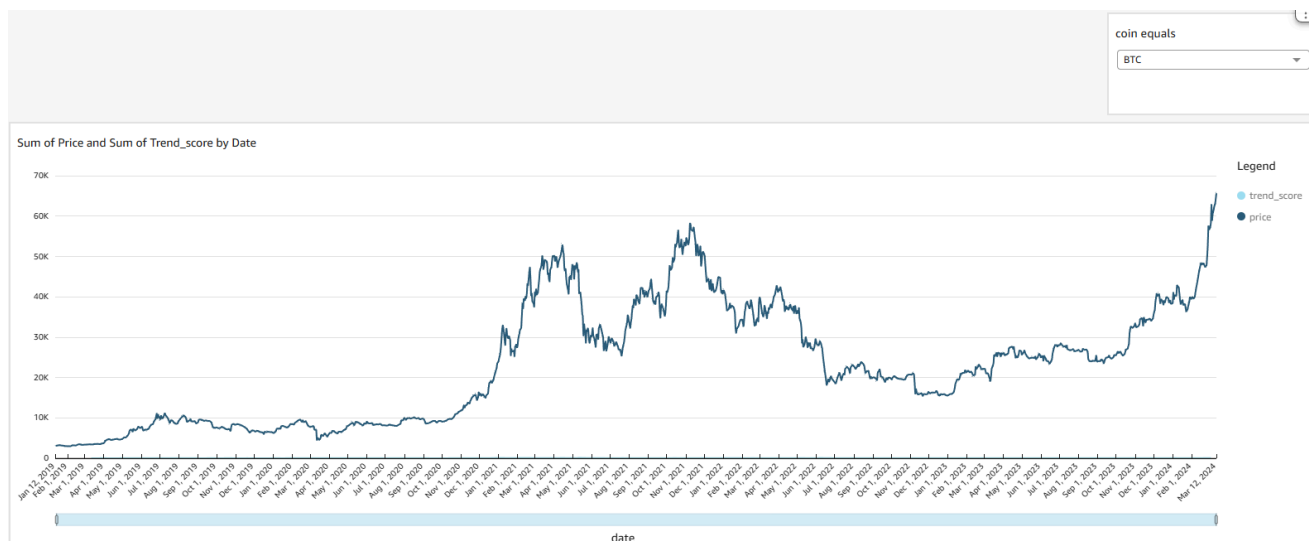


Figure 13: Correlazione temporale Bitcoin (BTC): confronto tra prezzo e interesse di ricerca su Google. Il trend score raggiunge valori più elevati e frequenti rispetto a Monero, riflettendo la maggiore esposizione mediatica di Bitcoin e una correlazione più marcata con le fasi di rialzo del prezzo.



Figure 14: Correlazione temporale Monero: confronto tra prezzo e interesse di ricerca su Google. A differenza di Bitcoin, il trend score mostra valori mediamente più bassi e meno frequenti, riflettendo una minore esposizione mediatica e una correlazione meno marcata con le variazioni di prezzo.

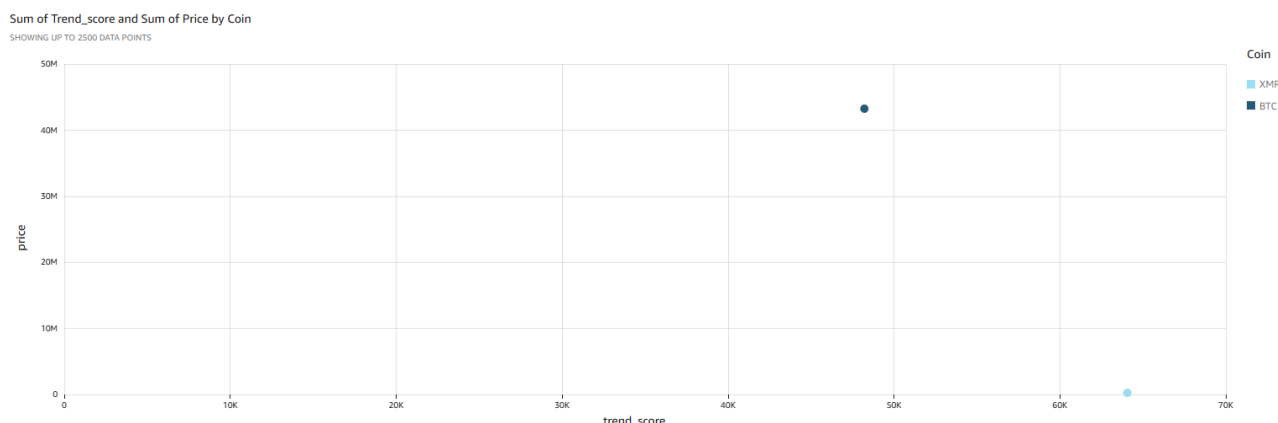


Figure 15: Analisi aggregata BTC vs XMR: scatter plot dei valori cumulativi di prezzo e trend score. Il grafico evidenzia la forte differenza di scala tra Bitcoin e Monero, con Bitcoin caratterizzato da valori di prezzo e interesse di ricerca significativamente superiori. La separazione netta dei punti conferma la diversa rilevanza di mercato e mediatica delle due criptovalute.